

Database Project Report 2  
CS6083 Spring 2026  
snickr: A Web-Based Collaboration System

Christine Wagner                      Hongyi Zeng  
caw561@nyu.edu                      hz3866@nyu.edu

April 24, 2026

## Contents

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                      | <b>2</b>  |
| <b>2</b> | <b>Database Schema Design</b>                                            | <b>2</b>  |
| 2.1      | ER Diagram . . . . .                                                     | 2         |
| 2.2      | Relational Schema . . . . .                                              | 2         |
| 2.3      | Normalization Analysis . . . . .                                         | 3         |
| 2.4      | Design Assumptions . . . . .                                             | 3         |
| 2.5      | Application-Level Enforcement . . . . .                                  | 4         |
| 2.6      | Space Efficiency . . . . .                                               | 5         |
| <b>3</b> | <b>Schema Creation</b>                                                   | <b>6</b>  |
| <b>4</b> | <b>Queries</b>                                                           | <b>7</b>  |
| 4.1      | (1) Create a new user account . . . . .                                  | 7         |
| 4.2      | (2) Create a new public channel (with authorization check) . . . . .     | 7         |
| 4.3      | (3) For each workspace, list all current administrators . . . . .        | 7         |
| 4.4      | (4) Pending invitations per public channel (older than 5 days) . . . . . | 8         |
| 4.5      | (5) List all messages in a channel in chronological order . . . . .      | 8         |
| 4.6      | (6) List all messages posted by a particular user . . . . .              | 9         |
| 4.7      | (7) Search accessible messages by keyword . . . . .                      | 9         |
| <b>5</b> | <b>Test Data</b>                                                         | <b>10</b> |
| 5.1      | Data Structure Overview . . . . .                                        | 10        |
| 5.2      | Sample Data Snapshot . . . . .                                           | 10        |
| <b>6</b> | <b>System Architecture</b>                                               | <b>12</b> |
| 6.1      | Technology Stack . . . . .                                               | 12        |
| 6.2      | Architecture Diagram . . . . .                                           | 12        |
| 6.3      | RPC API Design . . . . .                                                 | 12        |
| 6.4      | Available API Functions . . . . .                                        | 12        |

|           |                                                         |           |
|-----------|---------------------------------------------------------|-----------|
| <b>7</b>  | <b>Implementation Details</b>                           | <b>13</b> |
| 7.1       | Backend Implementation . . . . .                        | 13        |
| 7.2       | Frontend Implementation . . . . .                       | 13        |
| 7.3       | Key Features . . . . .                                  | 14        |
| <b>8</b>  | <b>Security</b>                                         | <b>15</b> |
| 8.1       | SQL Injection Prevention . . . . .                      | 15        |
| 8.2       | Cross-Site Scripting (XSS) Prevention . . . . .         | 15        |
| 8.3       | Cross-Site Request Forgery (CSRF) . . . . .             | 15        |
| 8.4       | Authentication and Session Security . . . . .           | 15        |
| <b>9</b>  | <b>Concurrency and Transaction Safety</b>               | <b>17</b> |
| 9.1       | Transaction Management . . . . .                        | 17        |
| 9.2       | Isolation Level and Concurrency Guarantees . . . . .    | 17        |
| 9.3       | Duplicate Prevention . . . . .                          | 18        |
| <b>10</b> | <b>Deployment</b>                                       | <b>19</b> |
| 10.1      | Docker Compose Configuration . . . . .                  | 19        |
| 10.2      | Startup Sequence . . . . .                              | 19        |
| 10.3      | Environment Variables . . . . .                         | 19        |
| <b>11</b> | <b>User Guide</b>                                       | <b>20</b> |
| 11.1      | Getting Started . . . . .                               | 20        |
| 11.2      | Main Interface . . . . .                                | 20        |
| 11.3      | Top Bar Actions . . . . .                               | 20        |
| 11.4      | Workspace Management . . . . .                          | 20        |
| 11.5      | Channel Management . . . . .                            | 21        |
| 11.6      | Messaging Features . . . . .                            | 21        |
| <b>12</b> | <b>Session Logs</b>                                     | <b>22</b> |
| 12.1      | Session 1: Registration and Workspace Setup . . . . .   | 22        |
| 12.2      | Session 2: Invitations and Collaboration . . . . .      | 22        |
| 12.3      | Session 3: Admin Operations . . . . .                   | 23        |
| 12.4      | Session 4: Direct Messaging . . . . .                   | 23        |
| 12.5      | Session 5: Message Search with Access Control . . . . . | 23        |
| <b>13</b> | <b>Design Decisions and Revisions for Project 2</b>     | <b>24</b> |
| 13.1      | Schema Revisions . . . . .                              | 24        |
| 13.2      | Design Decisions . . . . .                              | 24        |
| 13.3      | Known Limitations . . . . .                             | 24        |
| <b>14</b> | <b>Extra Features</b>                                   | <b>25</b> |
| <b>15</b> | <b>Conclusion</b>                                       | <b>25</b> |

# 1 Introduction

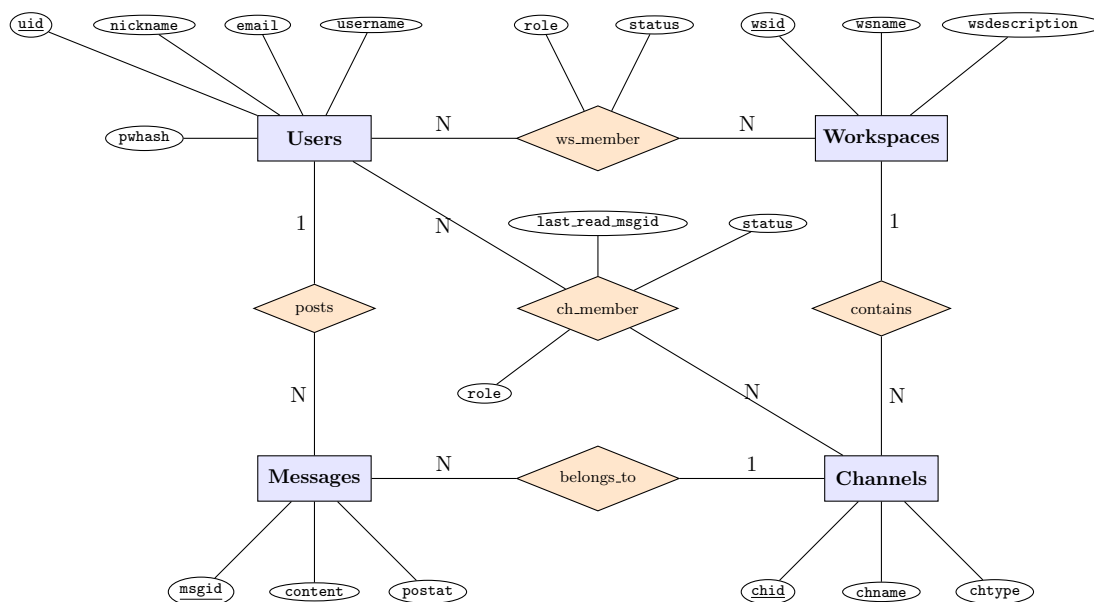
This report documents the design and implementation of *snickr*, a web-based collaboration system inspired by Slack. The system allows users to create workspaces, organize communication through public, private, and direct channels, and exchange messages within those channels.

The database is built on PostgreSQL 15 and consists of six relations: **users**, **workspaces**, **workspace\_members**, **channels**, **channel\_members**, and **messages**. Access control is enforced at the application level — a single DBMS account is used, and the application restricts content visibility based on workspace and channel membership.

This report covers both Project 1 (schema design and queries) and Project 2 (web interface implementation). Section 2 presents the database schema design with ER diagrams. Section 3 provides the schema creation SQL. Section 4 documents the seven required queries. Section 5 discusses the test data. Sections 6–12 cover the Project 2 web application: architecture, implementation details, security measures, concurrency handling, user guide, and sample session logs. Section 13 discusses design decisions and revisions made for Project 2. Section 14 describes additional features implemented for extra credit.

## 2 Database Schema Design

### 2.1 ER Diagram



### 2.2 Relational Schema

The ER diagram translates into the following six relations:

1. **users**(uid, email, username, nickname, pwhash, createdat, updatedat)
  - email is UNIQUE NOT NULL
2. **workspaces**(wsid, wsname, wsdescription, createdat, updatedat)
3. **workspace\_members**(wsid, uid, role, status, createdat, updatedat)

- $\text{wsid} \rightarrow \text{workspaces}(\text{wsid})$
  - $\text{uid} \rightarrow \text{users}(\text{uid})$
  - $\text{role} \in \{\text{creator}, \text{admin}, \text{member}\}$
  - $\text{status} \in \{\text{pending}, \text{accepted}, \text{rejected}\}$
4. **channels**(chid, wsid, cname, chtype, createdat, updatedat)
- $\text{wsid} \rightarrow \text{workspaces}(\text{wsid})$
  - $\text{chtype} \in \{\text{public}, \text{private}, \text{direct}\}$
5. **channel\_members**(chid, uid, role, status, last\_read\_msgid, createdat, updatedat)
- $\text{chid} \rightarrow \text{channels}(\text{chid})$
  - $\text{uid} \rightarrow \text{users}(\text{uid})$
  - $\text{role} \in \{\text{creator}, \text{member}\}$
  - $\text{status} \in \{\text{pending}, \text{accepted}, \text{rejected}\}$
6. **messages**(msgid, chid, uid, content, postat)
- $\text{chid} \rightarrow \text{channels}(\text{chid})$
  - $\text{uid} \rightarrow \text{users}(\text{uid})$

## 2.3 Normalization Analysis

All six relations are in **BCNF (Boyce–Codd Normal Form)**:

- **users**: The only candidate key is **uid** (surrogate). **email** is also a candidate key. Every non-trivial functional dependency has a superkey as its determinant.  $\Rightarrow$  BCNF.
- **workspaces**: Single candidate key **wsid**. All attributes depend on **wsid**.  $\Rightarrow$  BCNF.
- **workspace\_members**: Composite key (**wsid**, **uid**). The attributes **role**, **status**, **createdat**, **updatedat** are fully functionally dependent on the composite key. No partial or transitive dependencies.  $\Rightarrow$  BCNF.
- **channels**: Single candidate key **chid**. **wsid** is a foreign key, not a determinant of other attributes.  $\Rightarrow$  BCNF.
- **channel\_members**: Composite key (**chid**, **uid**). Same reasoning as **workspace\_members**.  $\Rightarrow$  BCNF.
- **messages**: Single candidate key **msgid**. All attributes depend on **msgid**.  $\Rightarrow$  BCNF.

## 2.4 Design Assumptions

- **A surrogate uid is used as the primary key for users**, rather than **email**. This allows users to change their email address without cascading updates across all foreign keys. The **email** column is constrained as **UNIQUE NOT NULL**.

- **Creator is stored in membership tables, not in workspace/channel tables.** The creator is the first member inserted with `role = 'creator'`. As specified in the requirements, the creator is treated as the initial administrator; query (3) therefore includes both `'creator'` and `'admin'` roles when listing administrators. Storing the creator as a role avoids redundancy and simplifies queries.
- **Invitations are tracked via the status field,** not as a separate entity. When a user is invited to a workspace or channel, a membership record is inserted with `status = 'pending'`. Upon acceptance, it is updated to `'accepted'`, and users may also explicitly decline an invitation by setting `'rejected'`. This design supports richer user interaction without introducing a separate invitations table.
- **Public channels are visible to all workspace members but still require explicit membership.** Users are not automatically added upon channel creation; instead, they must choose to join. When a user joins a public channel, a record is inserted into `channel_members` with `status = 'accepted'`, granting access to the channel's messages.
- **Passwords are stored as SHA-256 hashes.** The `pwhash` field stores a hashed password rather than plaintext for security.
- **Timestamps are recorded for all actions.** All tables include `createdat` and `updatedat` fields (messages use `postat` instead). While `updatedat` is not exercised by the required queries, it is included to support features such as profile edits, role changes, and invitation acceptance tracking.
- **Member removal** is supported by deleting the corresponding row from `workspace_members` or `channel_members`. No soft-delete flag is needed since the membership record itself represents the relationship.
- **Direct channels are limited to two members,** enforced at the application level rather than via a database constraint, since a `CHECK` on member count would require a trigger.
- **Message read state is tracked at the channel level.** Each `channel_members` record includes a `last_read_msgid` attribute, which stores the most recent message viewed by the user in that channel.
- **Access control is enforced at the application level,** not through database permissions or views, as specified in the project requirements.

## 2.5 Application-Level Enforcement

The following constraints are enforced by the web application rather than the database, either because SQL `CHECK` constraints cannot express them without triggers, or because they involve session state:

- **Direct channel membership limit.** The application ensures that exactly two users are associated with a direct channel at creation time.
- **Workspace invitation authority.** Only users with `role IN ('creator', 'admin')` in `workspace_members` are permitted to insert new membership records for other users.

- **Private channel invitation authority.** Only the channel creator (`role = 'creator'` in `channel_members`) may invite additional users to a private channel.
- **Session identification.** Each HTTP request includes a session cookie identifying the logged-in user `uid`. The application resolves this identity before executing any query.
- **Public channel join eligibility.** Before inserting a `channel_members` row for a public channel, the application verifies that the user has `status = 'accepted'` in the corresponding `workspace_members` relation.

## 2.6 Space Efficiency

- **No redundant creator columns.** Instead of storing a separate `creator_uid` in the `workspaces` and `channels` tables, we encode the creator as a membership record with `role = 'creator'`.
- **Composite primary keys** on `workspace_members` and `channel_members` eliminate the need for surrogate ID columns in junction tables.
- **VARCHAR with appropriate limits** is used instead of fixed-length `CHAR` fields, so shorter values do not waste space.
- **SERIAL auto-increment** is used for `uid`, `wsid`, `chid`, and `msgid` (4-byte integers), which is more compact than UUIDs.
- **No separate invitations table.** The `status` field on membership tables tracks invitation state in-place, avoiding an extra table and the joins it would require.

### 3 Schema Creation

The following SQL creates the schema in PostgreSQL with all key, foreign key, and check constraints:

```
CREATE TABLE users (
    uid          SERIAL          PRIMARY KEY,
    email        VARCHAR(255)    NOT NULL UNIQUE,
    username     VARCHAR(50)     NOT NULL,
    nickname     VARCHAR(50),
    pwhash       VARCHAR(255)    NOT NULL,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspaces (
    wsid         SERIAL          PRIMARY KEY,
    wsname       VARCHAR(100)    NOT NULL,
    wsdescription TEXT,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspace_members (
    wsid         INT             NOT NULL REFERENCES workspaces(wsid) ON DELETE CASCADE,
    uid          INT             NOT NULL REFERENCES users(uid) ON DELETE CASCADE,
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'admin', 'member')),
    status       VARCHAR(10)     NOT NULL CHECK (status IN ('pending', 'accepted', 'rejected'))
                                DEFAULT 'pending',
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (wsid, uid)
);

CREATE TABLE channels (
    chid         SERIAL          PRIMARY KEY,
    wsid         INT             NOT NULL REFERENCES workspaces(wsid) ON DELETE CASCADE,
    chname       VARCHAR(100),
    chtype       VARCHAR(10)     NOT NULL CHECK (chtype IN ('public', 'private', 'direct')),
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE channel_members (
    chid         INT             NOT NULL REFERENCES channels(chid) ON DELETE CASCADE,
    uid          INT             NOT NULL REFERENCES users(uid) ON DELETE CASCADE,
    role         VARCHAR(10)     NOT NULL CHECK (role IN ('creator', 'member')),
    status       VARCHAR(10)     NOT NULL CHECK (status IN ('pending', 'accepted', 'rejected'))
                                DEFAULT 'pending',
    last_read_msgid INT,
    createdat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updatedat    TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (chid, uid)
);

CREATE TABLE messages (
    msgid        SERIAL          PRIMARY KEY,
    chid         INT             NOT NULL REFERENCES channels(chid) ON DELETE CASCADE,
    uid          INT             NOT NULL REFERENCES users(uid) ON DELETE CASCADE,
    content      TEXT            NOT NULL,
    postat       TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

## 4 Queries

### 4.1 (1) Create a new user account

```
CREATE OR REPLACE PROCEDURE create_user(  
    p_email    VARCHAR,  
    p_username VARCHAR,  
    p_nickname VARCHAR,  
    p_pwhash   VARCHAR  
)  
LANGUAGE SQL  
AS $$  
    INSERT INTO users (email, username, nickname, pwhash)  
    VALUES (p_email, p_username, p_nickname, p_pwhash);  
$$;  
  
-- Example call:  
CALL create_user('gz1234@nyu.edu', 'gracezhu', 'Grace', 'a1b2c3d4e5f6');
```

**Test:** Calling with a new email succeeds and inserts a row into `users`. Calling with a duplicate email raises a unique constraint violation.

### 4.2 (2) Create a new public channel (with authorization check)

```
CREATE OR REPLACE PROCEDURE create_public_channel(  
    p_wsuid    INT,  
    p_uid      INT,  
    p_chname   VARCHAR  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_chid INT;  
BEGIN  
    IF NOT EXISTS (  
        SELECT 1 FROM workspace_members  
        WHERE wsid = p_wsuid AND uid = p_uid AND status = 'accepted'  
    ) THEN  
        RAISE EXCEPTION 'User % is not an accepted member of workspace %',  
            p_uid, p_wsuid;  
    END IF;  
    INSERT INTO channels (wsid, chname, chtype)  
    VALUES (p_wsuid, p_chname, 'public')  
    RETURNING chid INTO v_chid;  
    INSERT INTO channel_members (chid, uid, role, status)  
    VALUES (v_chid, p_uid, 'creator', 'accepted');  
END;  
$$;  
  
-- Example call:  
CALL create_public_channel(1, 1, 'announcements');
```

**Test:** Calling with an accepted workspace member succeeds; calling with a non-member raises an exception.

### 4.3 (3) For each workspace, list all current administrators

```
SELECT w.wsid, w.wsname, u.email, u.username, wm.role  
FROM workspaces w  
JOIN workspace_members wm ON w.wsid = wm.wsid  
JOIN users u ON wm.uid = u.uid  
WHERE wm.role IN ('creator', 'admin') AND wm.status = 'accepted'  
ORDER BY w.wsid, wm.role;
```



Result:

| wsid | wsname               | email          | username   | role    |
|------|----------------------|----------------|------------|---------|
| 1    | CS Department        | bs5678@nyu.edu | bobsmith   | admin   |
| 1    | CS Department        | az1234@nyu.edu | alicezhang | creator |
| 2    | Tandon Robotics Club | cw9012@nyu.edu | carolwang  | admin   |
| 2    | Tandon Robotics Club | bs5678@nyu.edu | bobsmith   | creator |
| 3    | DB Project Team      | gs6789@nyu.edu | gracesun   | admin   |
| 3    | DB Project Team      | cw9012@nyu.edu | carolwang  | creator |

#### 4.4 (4) Pending invitations per public channel (older than 5 days)

```
SELECT c.chid, c.chname, COUNT(cm.uid) AS pending_count
FROM channels c
LEFT JOIN channel_members cm ON c.chid = cm.chid
    AND cm.status = 'pending'
    AND cm.createdat < CURRENT_TIMESTAMP - INTERVAL '5 days'
WHERE c.wsid = 1 AND c.chtype = 'public'
GROUP BY c.chid, c.chname
ORDER BY c.chid;
```

Result:

| chid | chname   | pending_count |
|------|----------|---------------|
| 1    | general  | 1             |
| 4    | research | 0             |

#### 4.5 (5) List all messages in a channel in chronological order

```
CREATE OR REPLACE FUNCTION list_channel_messages(p_chid INT)
RETURNS TABLE (
    msgid INT, email VARCHAR, username VARCHAR,
    content TEXT, postat TIMESTAMP
)
LANGUAGE SQL
AS $$
    SELECT m.msgid, u.email, u.username, m.content, m.postat
    FROM messages m
    JOIN users u ON m.uid = u.uid
    WHERE m.chid = p_chid
    ORDER BY m.postat ASC;
$$;

-- Example call:
SELECT * FROM list_channel_messages(1);
```

Result (channel 1, #general — first 5 rows):

| msgid | email          | username   | content                                    | postat           |
|-------|----------------|------------|--------------------------------------------|------------------|
| 1     | az1234@nyu.edu | alicezhang | Welcome to the CS Department workspace!    | 2026-04-01 09:00 |
| 2     | bs5678@nyu.edu | bobsmith   | Thanks Alice! Excited to be here.          | 2026-04-01 09:05 |
| 3     | cw9012@nyu.edu | carolwang  | Quick question - has anyone seen...        | 2026-04-01 09:10 |
| 4     | az1234@nyu.edu | alicezhang | Not yet, but I heard they are adding...    | 2026-04-01 09:15 |
| 5     | gs6789@nyu.edu | gracesun   | Hi everyone! Just joined the department... | 2026-04-01 09:20 |

## 4.6 (6) List all messages posted by a particular user

```
SELECT m.msgid, c.chname, m.content, m.postat
FROM messages m
JOIN channels c ON m.chid = c.chid
WHERE m.uid = 1 -- Alice
ORDER BY m.postat DESC;
```

**Result (Alice, uid=1):** Returns all messages Alice has posted across #general, #hiring-committee, DM with Bob, and #research, ordered most recent first.

## 4.7 (7) Search accessible messages by keyword

```
CREATE OR REPLACE FUNCTION search_accessible_messages(
    p_uid INT, p_keyword VARCHAR
)
RETURNS TABLE (
    msgid INT, chname VARCHAR, author VARCHAR,
    content TEXT, postat TIMESTAMP
) LANGUAGE SQL AS $$
    SELECT m.msgid, c.chname, u.email AS author,
           m.content, m.postat
    FROM messages m
    JOIN channels c ON m.chid = c.chid
    JOIN users u ON m.uid = u.uid
    JOIN channel_members cm ON c.chid = cm.chid AND cm.uid = p_uid
    JOIN workspace_members wm
        ON c.wsid = wm.wsid AND wm.uid = p_uid
    WHERE cm.status = 'accepted'
        AND wm.status = 'accepted'
        AND m.content ILIKE '%' || p_keyword || '%'
    ORDER BY m.postat DESC;
$$;
-- Example call:
SELECT * FROM search_accessible_messages(1, 'database');
```

### Access control verification:

1. **Alice (uid=1):** Accepted in WS1 channels and WS3 channels  $\Rightarrow$  results from both workspaces. ✓
2. **Dave (uid=4):** Pending in channel #general but accepted in WS2 #events  $\Rightarrow$  only the #events match returned. ✓
3. **Eve (uid=5):** Pending workspace invitation  $\Rightarrow$  0 results. ✓

## 5 Test Data

The test dataset consists of **8 users**, **3 workspaces**, **11 channels** (covering public, private, and direct types), and **over 120 messages** with realistic conversation content.

### 5.1 Data Structure Overview

- **8 users** span three workspaces with overlapping membership. Bob, Carol, and Dave appear in multiple workspaces, testing cross-workspace queries.
- **3 workspaces:** CS Department (academic faculty), Tandon Robotics Club (student extracurricular), DB Project Team (course project). This variety creates realistic, distinguishable usage patterns.
- **Role diversity:** Alice is a workspace creator (WS1) and channel creator. Bob is both a workspace creator (WS2) and an admin (WS1). Carol is a workspace creator (WS3) and admin (WS2). This tests query (3).
- **Pending invitations:** Eve has a pending workspace invite; Dave and Frank have pending channel invites older than 5 days. This tests query (4).
- **120+ messages** are distributed across all 11 channels with natural conversation patterns covering topics such as curriculum planning, paper submissions, robotics competitions, hardware procurement, and database schema design.
- **Diverse channel types** per workspace: each workspace has public channels, and at least one private channel or direct message channel.

### 5.2 Sample Data Snapshot

Users:

| uid | email          | username    | nickname |
|-----|----------------|-------------|----------|
| 1   | az1234@nyu.edu | alicezhang  | Alice    |
| 2   | bs5678@nyu.edu | bobsmith    | Bob      |
| 3   | cw9012@nyu.edu | carolwang   | Carol    |
| 4   | dj3456@nyu.edu | davejohnson | Dave     |
| 5   | ec7890@nyu.edu | evechen     | Eve      |
| 6   | fl2345@nyu.edu | frankli     | Frank    |
| 7   | gs6789@nyu.edu | gracesun    | Grace    |
| 8   | hk1234@nyu.edu | henrykim    | Henry    |

Test Account Credentials:

| Username    | Password |
|-------------|----------|
| alicezhang  | alice123 |
| bobsmith    | bob456   |
| carolwang   | carol789 |
| davejohnson | dave012  |
| evechen     | eve345   |
| frankli     | frank678 |
| gracesun    | grace901 |
| henrykim    | henry234 |

Workspace Members (excerpt):

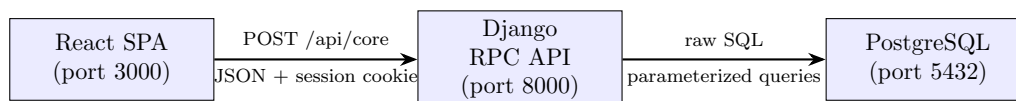
| wsid | uid | role    | status   | notes                     |
|------|-----|---------|----------|---------------------------|
| 1    | 1   | creator | accepted | Alice created CS Dept     |
| 1    | 2   | admin   | accepted | Bob is admin              |
| 1    | 5   | member  | pending  | Eve invited, not joined   |
| 2    | 2   | creator | accepted | Bob created Robotics      |
| 2    | 1   | member  | pending  | Alice invited, not joined |
| 3    | 3   | creator | accepted | Carol created DB Project  |
| 3    | 7   | admin   | accepted | Grace is admin            |

## 6 System Architecture

### 6.1 Technology Stack

- **Backend:** Django 5.2 (Python 3.11) — serves the API via a single RPC endpoint; all queries use raw SQL with parameterized placeholders (no ORM)
- **Database:** PostgreSQL 15 — relational backend with CHECK constraints, ON DELETE CASCADE, and stored procedures
- **Frontend:** React 19 with Tailwind CSS 3.4 — single-page application with functional components and hooks
- **Deployment:** Docker Compose — PostgreSQL container + Django container; frontend runs on the host via `npm start`

### 6.2 Architecture Diagram



### 6.3 RPC API Design

Rather than a RESTful API with multiple endpoints and HTTP verbs, we use a single **Remote Procedure Call (RPC)** endpoint at `POST /api/core`. Every request contains a JSON body:

```
{"function": "function_name", "args": [], "kwargs": {}}
```

The server dispatches to the named Python function, which executes the corresponding SQL queries and returns the result as JSON. This approach was chosen for simplicity — the backend is a single `views.py` file with a function registry, making it easy to add new operations without creating new URL patterns or view classes.

### 6.4 Available API Functions

The system provides **26 RPC functions** organized into six categories:

| Category       | Functions                                                                                                                                                                                                                                                                                     |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Auth (4)       | <code>register</code> , <code>login</code> , <code>logout</code> , <code>me</code>                                                                                                                                                                                                            |
| Workspaces (8) | <code>get_workspaces</code> , <code>create_workspace</code> , <code>delete_workspace</code><br><code>get_workspace_members</code> , <code>invite_to_workspace</code><br><code>respond_workspace_invite</code> , <code>update_workspace_member</code> , <code>leave_workspace</code>           |
| Channels (7)   | <code>get_channels</code> , <code>create_channel</code> , <code>delete_channel</code><br><code>get_channel_members</code> , <code>invite_to_channel</code> , <code>respond_channel_invite</code><br><code>join_channel</code> , <code>leave_channel</code> , <code>get_public_channels</code> |
| Messages (3)   | <code>get_messages</code> , <code>send_message</code> , <code>search_messages</code>                                                                                                                                                                                                          |
| Users (1)      | <code>search_users</code>                                                                                                                                                                                                                                                                     |
| Reports (2)    | <code>get_workspace_admins</code> , <code>get_pending_channel_invites</code>                                                                                                                                                                                                                  |
| Admin (1)      | <code>initialize</code>                                                                                                                                                                                                                                                                       |

## 7 Implementation Details

### 7.1 Backend Implementation

The entire backend is contained in `DBProject/views.py`. Key implementation choices:

- **Raw SQL only.** Every database operation uses `connection.cursor()` and parameterized SQL queries. Django's ORM is not used, in compliance with the course requirement. For example, the login function:

```
def login(request, username, password):
    with connection.cursor() as c:
        c.execute(
            "SELECT uid, email, username, nickname, pwhash "
            "FROM users WHERE username = %s",
            [username],
        )
        user = _dictfetchone(c)
    if not user or user["pwhash"] != _hash(password):
        return {"error": "invalid credentials"}
    request.session["uid"] = user["uid"]
    return user
```

- **Session-based authentication.** Django's built-in session framework stores the user's uid in a server-side session. A session cookie (`sessionid`) is sent to the browser. Every protected function checks `request.session.get("uid")` before proceeding.
- **Password hashing.** Passwords are hashed with SHA-256 (`hashlib.sha256`) before storage. The hash comparison is performed in Python, not in SQL.
- **Authorization checks.** Operations like `invite_to_workspace` first query the caller's role and verify it is 'creator' or 'admin' before proceeding.
- **Duplicate prevention.** Workspace and channel invitations use `INSERT ... ON CONFLICT (wsid, uid) DO NOTHING` to avoid duplicate membership records.

### 7.2 Frontend Implementation

The frontend is a React single-page application with the following component hierarchy:

- `App.js` — top-level routing (login vs. chat page); checks session on mount via `me()`
- `LoginPage.js` / `RegisterPage.js` — authentication forms with validation
- `ChatPage.js` — main layout with state management for workspaces, channels, messages
  - `WorkspaceList.js` — left sidebar listing workspaces with create button
  - `ChannelList.js` — middle sidebar listing channels with create/browse buttons
  - `ChatWindow.js` — message display with timestamps, message input with emoji picker
  - `Inbox.js` — dropdown for pending workspace and channel invitations
  - `SearchModal.js` — keyword search across all accessible messages
  - `CreateWorkspaceModal.js` — workspace creation with optional member invites

- `CreateChannelModal.js` — channel creation (public/private/direct)
- `WorkspaceMembersModal.js` — member management, role changes, member removal
- `ChannelSettingsModal.js` — channel member management and deletion
- `DiscoverChannelsModal.js` — browse and join public channels

All API calls go through `services/api.js`, which wraps the RPC protocol and includes `credentials: "include"` for session cookie handling across the cross-origin boundary (React on port 3000, Django on port 8000).

```
export const apiCall = async (func, args = [], kwargs = {}) => {
  const res = await fetch(BASE_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "include",
    body: JSON.stringify({ function: func, args, kwargs }),
  });
  const data = await res.json();
  if (!data.ok) throw new Error(data.error || "API error");
  return data.data;
};
```

## 7.3 Key Features

1. **User registration and login** — users create accounts with email, username, nickname, and password. Sessions persist across page refreshes via the `me` endpoint.
2. **Workspace management** — create workspaces, invite members (by searching usernames), promote/demote admins, remove members, leave or delete workspaces.
3. **Channel types** — public (discoverable, self-join), private (invite-only), and direct messages (two-person).
4. **Messaging** — send and view messages in real-time within a channel. Messages display chronologically with sender name and timestamp.
5. **Invitation inbox** — pending workspace and channel invitations appear in a dropdown; users can accept or reject each with a single click.
6. **Public channel discovery** — a “Browse” button lists public channels in the current workspace that the user has not yet joined. One-click join.
7. **Message search** — a search modal allows users to search for messages by keyword across all channels they have access to. Results show the channel name, author, content, and timestamp.
8. **Emoji picker** — an inline emoji picker with categorized Unicode emoji (Smileys, Gestures, Hearts, Objects, Food, Animals, Nature) allows users to insert emoji into messages.

## 8 Security

### 8.1 SQL Injection Prevention

All database queries use **parameterized queries** with `%s` placeholders. User-supplied values are never concatenated into SQL strings. Django's database adapter (`psycopg`) passes parameters separately to PostgreSQL, which treats them as data, not executable SQL.

```
# Safe: parameterized query
c.execute(
    "SELECT uid, email, username, nickname FROM users "
    "WHERE username ILIKE %s OR email ILIKE %s LIMIT 20",
    [f"%{query}%", f"%{query}%"],
)
```

The stored procedures defined in `query.sql` (e.g., `create_user`, `create_public_channel`) also use parameterized inputs, providing an additional layer of protection when called directly.

We verified that **no SQL string concatenation with user input** exists anywhere in the codebase. Every `c.execute()` call uses `%s` placeholders with a parameter list.

### 8.2 Cross-Site Scripting (XSS) Prevention

The frontend uses React, which **automatically escapes all content rendered in JSX**. When a variable is placed inside curly braces (`{msg.content}`), React converts special characters (`<`, `>`, `&`, `"`, `'`) to their HTML entity equivalents before inserting them into the DOM. This prevents any user-supplied content (message text, usernames, channel names) from being interpreted as HTML or JavaScript.

We do not use `dangerouslySetInnerHTML` anywhere in the codebase, so there is no bypass of React's built-in XSS protection.

### 8.3 Cross-Site Request Forgery (CSRF)

The RPC endpoint uses `@csrf_exempt` because the frontend is a separate React application that communicates via JSON POST requests with session cookies. CSRF protection is mitigated by:

1. The endpoint only accepts POST with a `Content-Type: application/json` header.
2. Browsers enforce CORS preflight checks for cross-origin JSON requests, preventing third-party sites from forging requests with the correct content type.
3. Django's `CORS_ALLOWED_ORIGINS` is configured to only allow the frontend's origin (`http://localhost:3000`).

### 8.4 Authentication and Session Security

- **Server-side sessions.** Django stores session data on the server (in the database by default). Only a session ID cookie is sent to the client.
- **Session cookie security.** The `sessionid` cookie is HTTP-only and signed by Django's secret key, preventing client-side JavaScript from reading or tampering with it.
- **Authentication guard.** Every protected RPC function checks `request.session.get("uid")` at the start. If the user is not logged in, an error is returned immediately.



- **Authorization checks.** Role-based operations (e.g., inviting users, promoting admins, deleting workspaces) verify the caller's role in the relevant membership table before proceeding.

## 9 Concurrency and Transaction Safety

### 9.1 Transaction Management

Multi-step database operations are wrapped in `transaction.atomic()` to ensure atomicity. If any step fails, the entire transaction is rolled back, preventing partial state changes.

The following operations use explicit transactions:

- `create_workspace` — inserts into `workspaces` and `workspace_members` atomically
- `create_channel` — checks membership, inserts into `channels` and `channel_members`
- `delete_channel` — deletes from `messages`, `channel_members`, and `channels`
- `delete_workspace` — cascading delete across five tables in dependency order
- `invite_to_workspace` — checks role, then inserts invitation
- `update_workspace_member` — checks role, then modifies/removes member
- `initialize` — drops and recreates all tables with test data

Example of a transactional multi-table delete:

```
def delete_workspace(request, wsid):
    uid = request.session.get("uid")
    if not uid:
        return {"error": "not logged in"}
    with transaction.atomic():
        with connection.cursor() as c:
            c.execute("SELECT role FROM workspace_members "
                      "WHERE wsid=%s AND uid=%s", [wsid, uid])
            row = _dictfetchone(c)
            if not row or row["role"] != "creator":
                return {"error": "only the workspace creator can delete it"}
            c.execute("DELETE FROM messages WHERE chid IN "
                      "(SELECT chid FROM channels WHERE wsid=%s)", [wsid])
            c.execute("DELETE FROM channel_members WHERE chid IN "
                      "(SELECT chid FROM channels WHERE wsid=%s)", [wsid])
            c.execute("DELETE FROM channels WHERE wsid=%s", [wsid])
            c.execute("DELETE FROM workspace_members WHERE wsid=%s", [wsid])
            c.execute("DELETE FROM workspaces WHERE wsid=%s", [wsid])
    return {"ok": True}
```

### 9.2 Isolation Level and Concurrency Guarantees

PostgreSQL's default isolation level (`READ COMMITTED`) ensures that concurrent sessions do not see uncommitted changes from other transactions. This prevents issues such as:

- A user reading messages from a channel that is being deleted by another user.
- Two users simultaneously creating a workspace and both getting the same `wsid` (prevented by `SERIAL`).
- An invitation being accepted while the workspace is being deleted.

Single-statement operations (e.g., `send_message`, `get_messages`) run in Django's default auto-commit mode, which is safe because a single SQL statement is inherently atomic in PostgreSQL.

### 9.3 Duplicate Prevention

The `INSERT ... ON CONFLICT DO NOTHING` pattern is used for invitations to prevent duplicate membership records when the same user is invited twice:

```
c.execute(
    "INSERT INTO workspace_members (wsid, uid, role, status) "
    "VALUES (%s, %s, 'member', 'pending') "
    "ON CONFLICT (wsid, uid) DO NOTHING",
    [wsid, invitee_uid],
)
```

This leverages the composite primary key `(wsid, uid)` as a natural uniqueness constraint.

## 10 Deployment

### 10.1 Docker Compose Configuration

The entire backend system runs with a single command: `docker compose up --build`.

```
# docker-compose.yml (simplified)
services:
  db:
    image: postgres:15
    env_file: .env
    ports: ["5432:5432"]
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./sqls/initialize.sql:/docker-entrypoint-initdb.d/01-init.sql
      - ./sqls/insert.sql:/docker-entrypoint-initdb.d/02-insert.sql

  web:
    build: .
    command: ./wait-for-db.sh
    ports: ["8000:8000"]
    depends_on: [db]
    env_file: .env
```

### 10.2 Startup Sequence

1. PostgreSQL container starts and auto-runs `initialize.sql` and `insert.sql` on first boot.
2. Django container waits for PostgreSQL to be ready (`wait-for-db.sh` polls port 5432).
3. Django runs `migrate` (for session tables) and starts the development server on port 8000.
4. The user runs `npm start` in the `frontend/` directory to start the React development server on port 3000.

### 10.3 Environment Variables

Database credentials are stored in a `.env` file (not committed to version control):

```
POSTGRES_DB=dbproject
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_HOST=db
```

## 11 User Guide

### 11.1 Getting Started

1. **Start the system:** Run `docker compose up --build` from the project root.
2. **Start the frontend:** In a separate terminal, run `cd frontend && npm start`.
3. **Open the app:** Navigate to `http://localhost:3000`.
4. **Log in or register:** Use a test account (e.g., `alicezhang / alice123`) or create a new account.

### 11.2 Main Interface

The main interface consists of three panels:

- **Left sidebar (purple):** Lists your workspaces. Click to select. The “+” button creates a new workspace.
- **Middle sidebar (gray):** Lists channels in the selected workspace. Click to open. Includes buttons for creating channels and browsing public channels.
- **Main area (white/gray):** Displays messages in the selected channel. Type a message and click Send or press Enter. An emoji picker button next to the input allows inserting Unicode emoji.

### 11.3 Top Bar Actions

- **Search:** Opens a modal where you can search for messages by keyword across all your accessible channels. Results show the channel name, author, content, and timestamp.
- **Inbox:** Shows pending workspace and channel invitations. Accept or reject each with a single click.
- **Logout:** Ends the session and returns to the login page.

### 11.4 Workspace Management

Click “Members & settings” at the bottom of the workspace sidebar to:

- View all workspace members and their roles
- Invite new members by searching usernames (creator/admin only)
- Promote or demote members to/from admin (creator/admin only)
- Remove members (creator/admin only)
- Leave the workspace (non-creators)
- Delete the workspace (creator only — cascades to all channels and messages)

## 11.5 Channel Management

Click “Channel settings” at the bottom of the channel sidebar to:

- View channel members
- Add members from the workspace (channel creator only, for private channels)
- Leave the channel
- Delete the channel (channel creator only)

## 11.6 Messaging Features

- **Send messages:** Type in the input field and press Enter or click Send.
- **Emoji:** Click the emoji button (smiley face) to open a picker with 7 categories (Smileys, Gestures, Hearts, Objects, Food, Animals, Nature). Click any emoji to insert it into the message.
- **Timestamps:** Each message displays the sender’s username and the time it was posted.
- **Search:** Use the Search button in the top bar to find messages containing a keyword across all channels you have access to.

## 12 Session Logs

Below are representative session logs demonstrating the core functionality of the system. Each log shows the API calls made and the system's responses.

### 12.1 Session 1: Registration and Workspace Setup

1. User opens the registration page and creates an account:
  - `register(email="jd1234@nyu.edu", username="janedoe", nickname="Jane", password="pass123")`
  - Response: `{uid: 9, email: "jd1234@nyu.edu", username: "janedoe"}`
2. User creates a new workspace:
  - `create_workspace(wsname="Study Group", wsdescription="Finals prep")`
  - Response: `{wsid: 4, wsname: "Study Group"}`
3. User creates a public channel in the workspace:
  - `create_channel(wsid=4, chname="algorithms", chtype="public")`
  - Response: `{chid: 12, chname: "algorithms", chtype: "public"}`
4. User sends a message:
  - `send_message(chid=12, content="Who is ready for the algorithms final?")`
  - Response: `{msgid: 121, postat: "2026-04-24T10:30:00"}`

### 12.2 Session 2: Invitations and Collaboration

1. Alice logs in and checks her inbox:
  - `login(username="alicezhang", password="alice123")`
  - `get_invitations() ⇒ [{type: "workspace", id: 2, name: "Tandon Robotics Club"}]`
2. Alice accepts the workspace invitation:
  - `respond_workspace_invite(wsid=2, accept=true)`
  - Alice can now see channels in the Robotics Club workspace.
3. Alice browses public channels and joins #events:
  - `get_public_channels(wsid=2) ⇒ [{chid: 5, chname: "events"}, {chid: 6, chname: "competitions"}]`
  - `join_channel(chid=5) ⇒ {ok: true}`
4. Alice searches for messages containing “robotics”:
  - `search_messages(keyword="robotics") ⇒` Returns results from #events in WS2 (now accessible after accepting the invite and joining).

### 12.3 Session 3: Admin Operations

1. Bob logs in and manages the CS Department workspace:
  - `login(username="bobsmith", password="bob456")`
  - `get_workspace_members(wsid=1)`  $\Rightarrow$  Lists all members with roles.
2. Bob invites Eve (who has a pending invite) — no duplicate is created due to `ON CONFLICT DO NOTHING`:
  - `invite_to_workspace(wsid=1, invitee_uid=5)`  $\Rightarrow$  `{ok: true}`
3. Bob promotes Carol to admin:
  - `update_workspace_member(wsid=1, target_uid=3, role="admin")`  $\Rightarrow$  `{ok: true}`

### 12.4 Session 4: Direct Messaging

1. Carol logs in and creates a DM with Henry:
  - `login(username="carolwang", password="carol789")`
  - `create_channel(wsid=3, chtype="direct", invitee_uid=8)`
  - Response: `{chid: 12, chtype: "direct"}`
2. Carol sends a direct message:
  - `send_message(chid=12, content="Henry, can you push the latest SQL changes?")`
  - Response: `{msgid: 122, postat: "2026-04-24T15:00:00"}`
3. Henry logs in and sees the DM:
  - `login(username="henrykim", password="henry234")`
  - `get_messages(chid=12)`  $\Rightarrow$  Returns Carol's message.

### 12.5 Session 5: Message Search with Access Control

1. Alice searches for "PostgreSQL":
  - `search_messages(keyword="PostgreSQL")`
  - Results include messages from WS3 #general (where she is a member) but **not** from WS2 channels (where her invite is still pending).
2. Dave searches for the same keyword:
  - `search_messages(keyword="PostgreSQL")`
  - Dave gets results only from WS2 channels where he is an accepted member. He does not see WS3 results because his WS3 invite is still pending.



## 13 Design Decisions and Revisions for Project 2

### 13.1 Schema Revisions

The schema from Project 1 was used without structural changes. The `ON DELETE CASCADE` constraints on foreign keys were already in place, and the `status` field on membership tables naturally supports the invitation workflow required by the web interface.

### 13.2 Design Decisions

- **RPC over REST.** A single endpoint simplifies both the backend (one URL pattern, one dispatcher) and the frontend (one `apiCall` helper). Adding a new feature requires only writing a new Python function and decorating it with `@rpc`.
- **Raw SQL only.** While Django provides a powerful ORM, we exclusively use raw SQL with parameterized queries to satisfy the course requirement and to maintain full control over the generated queries.
- **Session-based authentication over tokens.** Django's built-in session framework provides secure, server-side session management with minimal configuration. The session cookie is HTTP-only and signed, preventing client-side JavaScript from reading the session ID.
- **Application-level cascade deletes.** Although PostgreSQL supports `ON DELETE CASCADE`, the `delete_workspace` and `delete_channel` functions explicitly delete dependent rows in order. This gives us full control over the deletion order and allows us to wrap the entire operation in a transaction.
- **Docker Compose for deployment.** The entire system (Django + PostgreSQL) runs with a single `docker compose up --build` command, making it easy to set up for the demo.
- **Tailwind CSS for styling.** Utility-first CSS framework avoids the overhead of a component library while providing a polished, consistent UI.

### 13.3 Known Limitations

- No WebSocket support — the frontend does not receive real-time push updates. Users must refresh or re-select a channel to see new messages.
- No message editing or deletion after posting.
- No file or image uploads.
- Direct channel names are auto-generated as `DM - username` in the frontend but stored as `NULL` in the database.
- Passwords use SHA-256 hashing rather than a dedicated password-hashing algorithm (e.g., `bcrypt`). This is acceptable for a course project but not for production use.

## 14 Extra Features

The following features go beyond the basic requirements:

1. **Emoji Picker.** The chat input includes an inline emoji picker with 7 categories and over 120 Unicode emoji characters. Users can browse categories (Smileys, Gestures, Hearts, Objects, Food, Animals, Nature) and click to insert. The picker closes automatically when clicking outside.
2. **Message Search with Access Control.** The search function respects workspace and channel membership — users can only find messages in channels they have been accepted into. The search uses PostgreSQL’s `ILIKE` for case-insensitive matching.
3. **Public Channel Discovery.** A “Browse Channels” feature lets users discover and join public channels in their workspaces with a single click, similar to Slack’s channel browser.
4. **Invitation Inbox.** A unified inbox aggregates pending workspace and channel invitations with accept/reject buttons, providing a streamlined invitation management experience.
5. **User Search for Invitations.** When inviting members to a workspace, administrators can search for users by username or email with real-time results, rather than needing to know exact user IDs.
6. **Message Timestamps.** Every message displays both the sender’s name and the exact timestamp, formatted for readability.
7. **Docker Compose Deployment.** One-command setup with `docker compose up --build` — no manual database configuration required.

## 15 Conclusion

We have designed and implemented *snickr*, a fully functional web-based collaboration system backed by a PostgreSQL relational database. The system supports user registration, workspace and channel management, role-based access control, invitation workflows, messaging with emoji support, and keyword search — all through a clean React interface communicating with a Django backend via a single RPC endpoint.

Security measures include parameterized queries against SQL injection, React’s automatic XSS escaping, session-based authentication, CORS-restricted API access, and transactional consistency for multi-step operations. The system can be deployed locally via Docker Compose for a live demonstration.

The test dataset includes 8 users across 3 workspaces with 11 channels and over 120 messages, providing rich data for demonstrating all features during the demo.